

ExamAI: Adaptive Study Material Generation for Competitive Examinations Using Large Language Models

Rahul R. Bagewadi, Charan, Dakshath, Mahesh

Department of Data Science (Batch 2022–2026)

CMR University, Bengaluru, India

rahulrbagewadi@gmail.com

Abstract—Competitive examinations remain one of the primary vehicles for socioeconomic mobility in India, yet access to quality preparatory resources is heavily skewed toward urban, affluent demographics. Existing digital learning platforms address distribution but not *adaptation*: they deliver static video lectures and pre-compiled question banks that cannot respond to individual knowledge gaps, language preferences, or examination-specific syllabi.

This paper presents ExamAI, an open-architecture educational platform that employs large language models (LLMs) to generate syllabus-constrained study materials, structured quizzes, spaced-repetition flashcards, and context-locked tutoring—all natively produced in regional Indian languages without a secondary translation layer. The system is built on a serverless edge-computing stack (Next.js 16, Neon PostgreSQL, Google Gemini API) and incorporates a non-blocking income-certificate verification pipeline using multimodal OCR to automate subsidized access for Economically Weaker Section (EWS) candidates. We describe the architecture, the prompt-engineering strategies governing each generative module, and a cryptographic caching mechanism that curtails redundant API calls. Preliminary deployment results indicate sub-four-second content generation latency and accurate vernacular output, while also exposing practical bottlenecks in audio synthesis and complex-script PDF rendering that inform ongoing engineering effort.

Index Terms—Generative AI, Large Language Models, Adaptive Learning, Competitive Examinations, Multilingual Education, Prompt Engineering, Serverless Architecture

I. INTRODUCTION

Government service examinations in India—UPSC, SSC, RRB, state PSCs, and banking recruitment tests—collectively attract hundreds of millions of candidates every year. Performance in these examinations correlates strongly with access to structured preparatory material, yet the dominant delivery model remains the urban coaching institute, where annual fees routinely exceed INR 50,000. For candidates from Economically Weaker Sections, rural geographies, or vernacular-medium educational backgrounds, this creates a compounding disadvantage: the examinations designed to enable social mobility are themselves gated by socioeconomic privilege.

Digital platforms have partially addressed the distribution problem. Massive Open Online Courses (MOOCs) and mobile learning apps deliver content at scale, but their pedagogical model is fundamentally broadcast-oriented. A pre-recorded lecture on Indian Polity is identical whether the student is

a graduate-level UPSC aspirant in Delhi or a 10th-pass RRB candidate in rural Karnataka studying in Kannada. The content neither adapts to the target examination’s specific syllabus boundaries nor to the student’s demonstrated weaknesses.

Recent advances in generative AI—particularly the emergence of instruction-tuned LLMs with strong multilingual capabilities—open a path beyond this static paradigm. Rather than curating content libraries manually, it becomes feasible to *synthesize* study materials on demand, constrained to a specific examination syllabus, difficulty level, and language, at marginal per-query cost.

This paper describes ExamAI, a platform built around this premise. The core contributions are as follows:

- **Syllabus-locked generative modules** for notes, quizzes, flashcards, and doubt resolution, where each module enforces examination-specific content boundaries through structured prompt engineering rather than post-hoc filtering.
- **Native multilingual generation** that produces pedagogical content directly in the target vernacular (Kannada, Hindi, Tamil, Telugu, Malayalam, Urdu) by drawing on the LLM’s intrinsic language capabilities, thereby avoiding the fidelity loss inherent in pipeline translation.
- **Automated EWS verification** via a non-blocking multimodal OCR pipeline that extracts income data from uploaded certificates, assigns a fraud-risk score, and grants provisional subsidized access while queueing documents for administrative review.
- **A cryptographic content cache** that hashes prompt parameters to detect and serve duplicate queries without redundant LLM invocations, reducing both operational cost and response latency.

The remainder of this paper is organized as follows. Section II surveys related work in AI-assisted education. Section III details the system architecture. Section IV describes the methodology governing each generative module. Section V covers implementation specifics and key design decisions. Section VI presents preliminary results and discusses observed trade-offs. Sections VII and VIII address limitations and planned future work, respectively.

II. RELATED WORK

A. Learning Management Systems and MOOCs

The shift from physical classrooms to digital learning has unfolded over several generations. Early Learning Management Systems—Moodle, Blackboard—digitized course administration but replicated the instructor-broadcast model almost entirely. MOOCs (Coursera, edX, Khan Academy) subsequently solved the *reach* problem, delivering university-caliber lectures to millions worldwide [2]. Both paradigms, however, share a fundamental constraint: content is authored once and consumed uniformly. A student struggling with a specific concept receives the same material as one who has already mastered it. Where personalization exists at all, it is limited to adaptive sequencing of pre-authored modules rather than dynamic content generation.

B. Spaced Repetition and Active Recall Tools

Applications such as Anki [6] and Quizlet have convincingly demonstrated the efficacy of spaced repetition for long-term retention. Their limitation lies upstream: students must manually author or curate flashcard decks, a process that is both time-intensive and, ironically, presupposes the subject mastery the student is trying to acquire. This creates a bootstrapping problem—the students who stand to benefit most from structured revision are precisely those least equipped to create the revision materials.

C. LLM-Powered Educational Tools

The integration of LLMs into education is a rapidly expanding area. Duolingo Max employs GPT-4 for conversational language practice. Khanmigo, developed by Khan Academy in collaboration with OpenAI [7], provides a Socratic tutoring interface across STEM subjects. These systems validate the viability of LLM-based pedagogy but exhibit three characteristics that limit their applicability to the Indian competitive examination context:

- 1) **Generalized scope.** They target broad educational use rather than specific examination syllabi with defined content boundaries.
- 2) **Limited vernacular depth.** Multilingual support, where offered, typically relies on translation APIs applied after English-language generation—a pipeline that degrades idiomatic accuracy and introduces additive latency.
- 3) **No socioeconomic access model.** None incorporate mechanisms for means-tested subsidization or automated income verification.

ExamAI addresses these gaps by coupling strict generative boundaries (examination-specific prompt constraints) with native multilingual output and an integrated EWS verification workflow.

III. SYSTEM ARCHITECTURE

A. Design Principles

Three architectural priorities guided the system design. First, **latency minimization**: competitive exam aspirants often

study in fragmented time windows—commute hours, lunch breaks, gaps between classes—so content must arrive in seconds, not minutes. Second, **elastic scalability**: study activity peaks sharply in the weeks before examination dates, requiring infrastructure that scales without manual provisioning. Third, **cost proportionality**: the platform targets underserved demographics, so operational costs must scale linearly with actual usage rather than demanding fixed infrastructure investment upfront.

These considerations led to a serverless, edge-first architecture.

B. Component Overview

Frontend. The user interface is built with Next.js 16 (App Router) and React 19. The design prioritizes minimal client-side JavaScript by making extensive use of React Server Components (RSC)—UI elements that render entirely on the server and stream pre-built HTML to the client. This reduces the JavaScript bundle delivered to the browser, an important consideration given that many target users access the platform on low-end mobile devices over bandwidth-constrained networks.

Backend. Server-side logic runs on Node.js edge runtimes via Next.js Server Actions. This pattern eliminates the need for traditional REST API endpoints exposed to the client; instead, the frontend invokes server-side functions directly through React’s action mechanism, with authentication, payload validation, and rate limiting enforced at the server boundary. In practice, this yields a smaller attack surface and substantially less boilerplate compared to a conventional API gateway.

Database. Persistent storage uses Neon Serverless PostgreSQL [4], which provides connection pooling at the edge—a necessity in serverless environments where each function invocation would otherwise open and close a fresh database connection. The data access layer uses Drizzle ORM [8], selected for its compile-time type safety with TypeScript schemas. This catches schema-data mismatches at development time rather than at runtime in production.

AI Services. The entire generative engine—text generation, multimodal OCR, and audio synthesis—is powered by the Google Gemini API [1]. To ensure the system remains cost-free to operate, all modules exclusively utilize the free tier of the Gemini Flash series. Text and JSON content generation use the `gemini-flash-latest` model with a fallback chain to `gemini-flash-lite-latest`. The EWS verification pipeline (Section V-C) similarly uses the Flash model for OCR. Audio synthesis uses the same Gemini API with `responseModalities` set to `AUDIO` and a prebuilt neural voice, consolidating all AI capabilities under a single free-tier API key.

C. Request Lifecycle

A representative request proceeds as follows. A student selects an examination (e.g., UPSC), a topic (e.g., “Gupta Empire”), a content type (e.g., notes), and a language (e.g., Hindi). The frontend dispatches a Server Action carrying these

parameters. The server-side handler first verifies the user’s authentication state and remaining generation credits. It then constructs a deterministic SHA-256 hash from the normalized request parameters and queries the `content_cache` table. On a cache hit, previously generated content is returned immediately—typically in under 200 milliseconds.

On a cache miss, the handler constructs a structured prompt (the design of which is detailed in Section IV), dispatches it to the Gemini API, validates the response against a predefined JSON schema using Zod [9], persists both the content and the cache entry to PostgreSQL, and streams the result back to the client. This full cycle completes in under four seconds for a typical notes generation request.

IV. METHODOLOGY

Each generative module in ExamAI addresses a distinct pedagogical task. The modules share a common prompting infrastructure—system-level instructions that establish the LLM’s role, output format constraints, and content boundaries—but differ in their output schemas and educational objectives.

A. Study Material Generation

Notes generation employs a few-shot prompting strategy in which the system prompt positions the LLM as a subject-matter expert for the specified examination. The prompt includes explicit boundary instructions: for a foundational clerical exam (e.g., RRB Group D), the model is directed to avoid graduate-level theoretical depth and instead emphasize factual recall and pattern-based reasoning. For an advanced examination like UPSC Mains, the same topic prompt permits analytical depth, cross-referencing between subjects, and essay-style exposition.

The rationale for this boundary-aware approach—rather than generating at a uniform difficulty and filtering afterward—is twofold. First, it avoids wasting LLM tokens on content that will ultimately be discarded. Second, and more importantly, the *framing* of explanations differs across difficulty levels. A concept explained for a clerical exam should use simpler analogies and shorter sentences; it should not merely be a truncated version of an advanced explanation.

B. Quiz Generation with Schema Enforcement

Unstructured quiz generation from LLMs frequently produces malformed output: missing answer keys, ambiguous phrasing, or factually unsupported distractors. ExamAI mitigates this through strict JSON schema enforcement at the prompt level. The LLM is instructed to return a JSON array where each element contains five mandatory fields:

- 1) `question`—the question stem.
- 2) `correct_answer`—the verified correct response.
- 3) `distractors`—an array of exactly three plausible but incorrect alternatives.
- 4) `rationale`—an explanation of why the correct answer holds and why each distractor does not.

- 5) `difficulty`—a categorical label (easy, medium, hard) aligned to the target examination.

The response is validated at runtime using Zod schema validation. Malformed responses trigger an automatic retry with an appended correction instruction rather than failing silently. In practice, first-attempt schema compliance exceeds 90% with the Gemini Flash model; the retry mechanism resolves remaining cases within one additional attempt.

C. Context-Locked Tutoring

The AI Tutor module implements what we term *context-window locking*—a prompt-engineering strategy that constrains the LLM’s conversational scope to the student’s selected examination and topic. The system prompt explicitly prohibits the model from answering questions outside the selected subject domain, generating code or completing assignments, and engaging in casual conversation or opinion-based discussion.

The design intent is to preserve educational utility while preventing two failure modes commonly observed in general-purpose LLM tutors: **prompt injection**, where users manipulate the model into ignoring its instructions, and **resource waste**, where off-topic queries consume API credits without educational value. The locking is implemented through system-level instructions rather than input filtering, which preserves conversational fluidity within the permitted domain.

We acknowledge a limitation: sufficiently adversarial users can circumvent prompt-level constraints. The approach is designed for a cooperative student population rather than a hostile threat model. Formal adversarial evaluation of this mechanism remains future work.

D. Flashcard Generation

Flashcard creation targets the active recall learning modality. The LLM receives a macro-topic and is instructed to decompose it into atomic, self-contained question-answer pairs—each pair should test exactly one fact or concept and be comprehensible without reference to other cards. This atomicity constraint is critical for spaced repetition: a card that conflates multiple concepts cannot be accurately scored as “known” or “unknown,” undermining the scheduling algorithm’s utility.

The generation prompt further instructs the model to prioritize high-yield facts—those with a high probability of appearing in the target examination based on historical question patterns described in the prompt context.

E. Native Multilingual Generation

A conventional approach to multilingual content would generate material in English and then apply a translation API (e.g., Google Translate) as a post-processing step. This pipeline suffers from two compounding problems: additional latency from a second API call per content item, and degraded idiomatic quality—particularly for educational content where terminology carries domain-specific meaning. The Hindi term for “fundamental rights” in a constitutional law context (*maulik adhikar*) differs from a literal translation of

the English phrase, and such distinctions matter in examination preparation.

ExamAI bypasses translation entirely. The system prompt instructs the LLM to generate content *directly* in the target language using native script. Modern LLMs such as Gemini are trained on multilingual corpora that include substantial Indian-language educational and governmental text, enabling them to produce output with natural phrasing and appropriate domain terminology—substantially better than pipeline translation for pedagogical content.

The supported languages—Hindi, Kannada, Tamil, Telugu, Malayalam, and Urdu—were selected based on the linguistic distribution of competitive examination candidates across Indian states.

F. Audio Content Synthesis

For students who prefer auditory learning or lack time for sustained reading, ExamAI provides server-side audio synthesis using the same Gemini API that powers content generation. The server-side TTS route dispatches the generated vernacular text to the Gemini model with `responseModalities` configured to `AUDIO` and a `speechConfig` specifying a prebuilt neural voice. The model returns raw PCM audio data, which the server wraps in a standard WAV header (24 kHz, 16-bit, mono) and serves to the client via the HTML5 `<audio>` element.

A key architectural decision was to use the same Gemini API for both text and audio generation rather than introducing a separate TTS service. This consolidates all AI operations under a single API key and billing account, simplifies deployment (no additional credentials or service integrations), and ensures that the TTS engine benefits from the same multilingual training data that produces high-quality vernacular text. The trade-off is that Gemini’s audio output modality is a newer feature with occasional latency spikes for longer texts, discussed further in Section VII.

V. IMPLEMENTATION DETAILS

A. Technology Stack

The implementation draws on the React ecosystem throughout. Next.js handles routing, server-side rendering, and the Server Actions pattern. TailwindCSS manages responsive styling. Runtime data validation uses Zod [9], a TypeScript-first schema validation library that provides both compile-time type inference and runtime validation—a property that proves particularly valuable when validating LLM outputs, which remain structurally unpredictable despite prompt constraints.

Authentication and user management are handled by Clerk [5], providing OAuth-based sign-in, session management, and integration with the credit-based access control system.

B. Database Schema

The PostgreSQL schema follows a normalized relational design. A central `users` table maintains one-to-many relationships with two primary content tables:

- `generated_content`—stores the output of each generative module as JSONB blobs, tagged with metadata (examination, topic, language, content type, generation timestamp).
- `wellness_logs`—tracks self-reported daily mood and study consistency metrics, intended for future integration with an adaptive learning algorithm.

A separate `content_cache` table implements the caching layer. Each row stores a SHA-256 hash of the normalized prompt parameters alongside the corresponding generated content. On incoming requests, the server computes the hash from current parameters and performs an index lookup; a match returns cached content without invoking the LLM. In testing across a simulated multi-user workload, approximately 30% of requests were served from cache—a figure expected to increase with user population as popular examination-topic-language combinations converge.

C. EWS Verification Pipeline

A distinctive design decision was implementing the EWS income-certificate verification as a *non-blocking* process. A conventional approach would gate platform access behind manual document review, introducing delays of days or weeks—unacceptable for a student whose examination date is imminent. The pipeline instead operates in five stages:

- 1) The student uploads a photograph or scan of their income certificate.
- 2) The server dispatches the image to Gemini Vision with an OCR-oriented prompt that extracts the declared annual income, the certificate holder’s name, and the issuing authority.
- 3) A rule-based scoring function evaluates the extracted data against expected ranges and formats, producing a numerical confidence score.
- 4) If the confidence score exceeds a configurable threshold, the student receives *immediate* provisional subsidized access. The document is simultaneously queued for manual administrative review.
- 5) If the score falls below the threshold, access is withheld pending manual verification.

This design optimizes for the student’s learning momentum. A first-generation examination candidate who has gathered the documentation to apply for subsidized access should not face a multi-day bureaucratic delay. The trade-off is a window of potential abuse: a fraudulent certificate that happens to score above the confidence threshold would receive temporary access before manual review catches it. Given the educational context and the modest monetary value of the subsidy, we consider this an acceptable risk.

VI. RESULTS AND DISCUSSION

A. Content Generation Performance

Preliminary testing across a range of examinations (UPSC Prelims, SSC CGL, RRB NTPC, Karnataka PSC) and languages (English, Hindi, Kannada) shows that the platform consistently generates syllabus-aligned study materials within

a four-second latency envelope for notes and flashcards. Quiz generation, which demands structured JSON output with multiple interdependent fields, exhibits higher variance—typical latency ranges from three to seven seconds depending on the number of questions requested.

Content quality, assessed through manual review by domain-knowledgeable evaluators, was rated as examination-appropriate in the majority of cases. Notes for factual-recall examinations (SSC, RRB) were particularly strong, producing concise, well-organized summaries with accurate factual content. Notes targeting analytical examinations (UPSC Mains) occasionally lacked the depth of nuance expected in essay-format answers, suggesting that prompt engineering for open-ended analytical content requires further refinement.

B. Multilingual Output Quality

Native vernacular generation produced substantially more natural output than a baseline comparison using English generation followed by the Google Translate API. Domain-specific terminology was handled correctly in most cases: the model used established Hindi and Kannada terms for constitutional, historical, and scientific concepts rather than producing awkward transliterations or calques.

This advantage was most pronounced for Hindi and Kannada, where the LLM’s training data likely includes substantial educational text. For less-resourced languages—Malayalam and Urdu in particular—output quality was acceptable but occasionally exhibited unnatural phrasing or code-switching to English for technical terms. This disparity reflects the uneven representation of Indian languages in current LLM training corpora and is not something addressable at the application layer.

C. Context-Locking Efficacy

The context-locked chatbot successfully deflected off-topic queries during informal testing. Requests for code generation, opinions on current events, and casual conversation were declined with appropriate redirections to the study topic. However, this evaluation involved cooperative testers rather than adversarial users. The robustness of prompt-level context locking against determined prompt injection remains an open question.

D. Caching Effectiveness

The content cache reduced LLM API invocations by approximately 30% in a simulated workload where multiple users studied overlapping examination topics. Cache hit rates varied by subject: high-demand topics in popular examinations (e.g., Indian Polity for UPSC) showed hit rates above 45%, while niche topics approached zero. The aggregate hit rate is expected to improve as the user base grows and query distributions stabilize.

VII. LIMITATIONS

The current system exhibits several limitations that constrain its production readiness.

API Dependency. The platform’s generative capabilities depend entirely on the availability, latency, and pricing stability of the Google Gemini API, which powers text generation, OCR, and audio synthesis. An outage or rate-limit change would degrade or disable all core functionality simultaneously. While consolidating on a single provider simplifies operations, it also concentrates risk. Mitigating this through self-hosted model inference or multi-provider redundancy is a priority but introduces significant infrastructure complexity and cost that the current project scope does not accommodate.

Audio Synthesis Latency. Generating audio for long-form notes (2,000+ words) introduces a perceptible time-to-first-byte delay, typically 5–8 seconds, which disrupts an otherwise responsive user experience. Streaming audio generation—producing and playing audio incrementally rather than buffering the complete file—would address this but requires a WebSocket or Server-Sent Events architecture not currently implemented.

Complex Script Rendering. The browser-native HTML-to-PDF conversion pipeline struggles with complex ligatures and conjunct characters in certain Indian scripts, particularly Malayalam and, to a lesser extent, Kannada. Exported PDFs occasionally display typographical artifacts—missing ligatures, incorrect character joining—that are absent in on-screen rendering. Resolving this likely requires server-side PDF generation with explicit OpenType font shaping support.

Absence of Formal Evaluation. The results reported in Section VI are based on informal testing and manual review rather than controlled experiments with measurable learning outcomes. A rigorous evaluation would require pre- and post-test designs with a representative student cohort—an undertaking planned for subsequent phases of this project but not yet conducted.

VIII. FUTURE WORK

Several extensions are planned to address the limitations above and to expand the platform’s capabilities.

Offline Access via Progressive Web App. Many target users study in environments with intermittent connectivity—rural areas, commuter trains, regions with limited mobile data coverage. A PWA with IndexedDB-backed local storage would allow previously generated content to be cached on-device for offline access. This is arguably the single highest-impact feature for the intended user demographic.

Mobile Application. A React Native port would unlock native mobile APIs for background audio playback (enabling study-while-commuting workflows) and push-notification-driven spaced repetition reminders—capabilities that are unreliable in browser-based applications.

Conversational Voice Tutoring. The current TTS pipeline is unidirectional: the system speaks, the student listens. A WebRTC-based bidirectional voice interface would allow students to pose questions verbally and receive spoken responses—a modality better suited to students more comfortable with oral communication than text input.

Adaptive Remediation. Tracking individual quiz performance over time would enable the system to identify persistent knowledge gaps and automatically generate targeted remedial content—flashcards focused specifically on concepts the student has repeatedly answered incorrectly. This closes the feedback loop between assessment and content generation, moving the platform toward genuinely adaptive learning.

IX. CONCLUSION

ExamAI demonstrates that generative AI, when combined with deliberate architectural choices—serverless edge computing, native multilingual generation, prompt-level content constraints, and automated socioeconomic verification—can deliver personalized, examination-specific study materials at marginal cost. The platform does not claim to replace structured coaching or mentorship; its aim is narrower but still consequential: ensuring that students who lack access to those resources are not left entirely without alternatives.

The system in its current form is a functional prototype that validates the core technical premise. LLMs can generate curriculum-appropriate, multilingual educational content with sufficient accuracy and speed for interactive use. The limitations identified—API dependency, audio latency, complex-script rendering, and the absence of formal efficacy measurement—are engineering and methodological challenges rather than fundamental barriers. Addressing them is the focus of ongoing work.

If platforms built on these principles can mature to the point where a student in a village in North Karnataka, studying in Kannada on a budget smartphone, receives study materials comparable in quality to those available at a premium coaching center in Delhi, the underlying technology will have justified its development.

REFERENCES

- [1] Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models,” *arXiv preprint arXiv:2312.11805*, Dec. 2023.
- [2] Vercel Inc., “Next.js: The React Framework for the Web,” [Online]. Available: <https://nextjs.org>. [Accessed: Apr. 2026].
- [3] Google, “Generate speech (text-to-speech),” *Gemini API Documentation*, [Online]. Available: <https://ai.google.dev/gemini-api/docs/audio>. [Accessed: Apr. 2026].
- [4] Neon Inc., “Neon—Serverless Postgres,” [Online]. Available: <https://neon.tech>. [Accessed: Apr. 2026].
- [5] Clerk Inc., “Clerk—Authentication and User Management,” [Online]. Available: <https://clerk.com>. [Accessed: Apr. 2026].
- [6] D. Elmes, “Anki—Intelligent Flashcards,” [Online]. Available: <https://apps.ankiweb.net>. [Accessed: Apr. 2026].
- [7] Khan Academy, “Khanmigo: Khan Academy’s AI-Powered Teaching Assistant,” 2023. [Online]. Available: <https://blog.khanacademy.org/tag/khanmigo/>. [Accessed: Apr. 2026].
- [8] Drizzle Team, “Drizzle ORM—TypeScript ORM for SQL Databases,” [Online]. Available: <https://orm.drizzle.team>. [Accessed: Apr. 2026].
- [9] C. Burda, “Zod—TypeScript-first Schema Validation,” [Online]. Available: <https://zod.dev>. [Accessed: Apr. 2026].